

STREAMBUF_REFERENCE

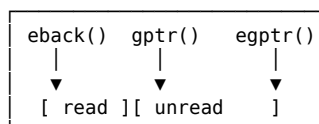
Core Definition

`<streambuf>` defines `basic_streambuf`, the buffer layer beneath every stream. A stream (`ostream`, `ifstream`, `stringstream`) handles *formatting*; the `streambuf` handles *transport* — moving characters between the program and a device via get and put areas. Deriving from `basic_streambuf` is how you make `operator<<` write to anything: a socket, a compressor, a tee, a null sink.

```
#CPP #STREAMBUF #FILEBUF #STRINGBUF #CUSTOM-STREAMS #BUFFERING #RDBUF
```

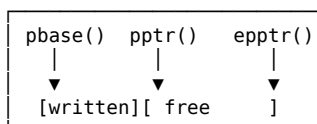
THE BUFFER MODEL

GET AREA (input)



eback = start of buffer
gptra = next char to read
egptr = one past the last valid char

PUT AREA (output)



pbase = start of buffer
pptr = next write position
epptr = one past the buffer end

gptra == epptr → get area exhausted → `underflow()` refills
pptr == epptr → put area full → `overflow()` drains

COMPLETE STREAMBUF QUICK REFERENCE

```
// =====
// PUBLIC INTERFACE – what streams call (the "pub" prefix)
// =====
sb.pubimbue(loc)           // locale      | Set locale          | Returns previous locale
sb.getloc()                // none      | Get locale          | Returns locale
sb.pubsetbuf(s, n)         // ptr + n   | Supply external buffer | Returns streambuf*
sb.pubseekoff(off, dir[, which]) // offset   | Seek relative       | Returns pos_type
sb.pubseekpos(pos[, which]) // position  | Seek absolute       | Returns pos_type
sb.pubsync()               // none      | Flush to device      | Returns int (0 = ok)

// =====
// PUBLIC – GET AREA (input)
// =====
sb.in_avail()              // none      | Chars readable now   | Returns streamsize
sb.snextc()                // none      | Advance THEN read    | Returns int_type
sb.sbumpc()                // none      | Read THEN advance    | Returns int_type
sb.sgetc()                 // none      | Read, do NOT advance | Returns int_type
sb.sgetn(s, n)             // ptr + n   | Read up to n chars   | Returns streamsize
sb.sputbackc(c)            // char      | Put back this char   | Returns int_type
sb.sungetc()               // none      | Back up one char     | Returns int_type

// =====
// PUBLIC – PUT AREA (output)
// =====
sb.sputc(c)                // char      | Write one char       | Returns int_type (eof on failure)
sb.sputn(s, n)             // ptr + n   | Write n chars        | Returns streamsize written

// =====
```

```

// PROTECTED – GET AREA POINTERS (for derived classes)
// =====
eback()           // none      | Start of get area   | Returns char_type*
gptr()            // none      | Next read position  | Returns char_type*
egptr()           // none      | End of get area     | Returns char_type*
gbump(n)          // int       | Advance gptr by n   | Returns void
setg(begin, next, end) // 3 ptrs   | Set all get pointers | Returns void

// =====
// PROTECTED – PUT AREA POINTERS
// =====
pbase()           // none      | Start of put area   | Returns char_type*
pptr()            // none      | Next write position  | Returns char_type*
epptr()           // none      | End of put area     | Returns char_type*
pbump(n)          // int       | Advance pptr by n   | Returns void
setp(begin, end)   // 2 ptrs   | Set put pointers    | Returns void (pptr = begin)

// =====
// PROTECTED VIRTUALS – OVERRIDE THESE
// =====
virtual int_type underflow() // Get area empty; refill it. Return the next char
                             // WITHOUT advancing gptr, or traits::eof().
virtual int_type uflow()    // Like underflow but DOES advance. Default calls
                             // underflow then gbump(1); override only for
                             // unbuffered sources.
virtual int_type overflow(c) // Put area full (or c to write unbuffered). Drain the
                             // buffer, write c, return anything != eof() on success.
virtual int_type pbackfail(c) // putback requested but no room / different char.
                             // Return eof() to refuse.
virtual int sync()          // Flush the put area to the device. Return 0 on success.
virtual streamsize showmanyc() // Chars available without blocking. -1 = definitely none.
virtual streamsize xsgetn(s, n) // Bulk read – override for a faster path
virtual streamsize xspn(s, n) // Bulk write – override for a faster path
virtual pos_type seekoff(off, dir, which) // Relative seek. Default: fails.
virtual pos_type seekpos(pos, which) // Absolute seek. Default: fails.
virtual streambuf* setbuf(s, n) // Accept an external buffer. Default: ignores.
virtual void imbue(loc) // React to a locale change. Default: no-op.

// =====
// CHAR_TRAITS – the vocabulary of buffer code
// =====
using Traits = std::char_traits<char>;
Traits::eof() // The end-of-file int_type value
Traits::to_int_type(c) // char → int_type (needed before returning a char)
Traits::to_char_type(i) // int_type → char
Traits::eq_int_type(a, b) // Compare int_types (handles eof correctly)
Traits::not_eof(i) // i if i != eof, else some non-eof value
Traits::length(s) / copy / move / assign / compare // Bulk char operations

// =====
// STANDARD DERIVED BUFFERS
// =====
std::filebuf // <fstream> – file-backed
std::stringbuf // <sstream> – string-backed
std::spanbuf // <spanstream> C++23 – fixed external buffer
std::syncbuf // <syncstream> C++20 – thread-safe accumulation

// =====
// TYPE ALIASES
// =====
std::streambuf // = basic_streambuf<char>
std::wstreambuf // = basic_streambuf<wchar_t>

```

COMMON PATTERNS & EXAMPLES

Buffer-to-Buffer Copy (the idiom worth knowing)

```
#include <fstream>
#include <iostream>

std::ifstream in("src.txt");
std::ofstream out("dst.txt");

out << in.rdbuf();           // Copies the whole stream – no loop, no temporary string
std::cout << in.rdbuf();     // Dump a file to the console

// This works because ostream has an operator<< overload taking streambuf*.
```

Redirecting a Standard Stream

```
#include <fstream>
#include <iostream>

class CoutRedirect {
    std::streambuf* old_;
public:
    explicit CoutRedirect(std::streambuf* newBuf) : old_(std::cout.rdbuf(newBuf)) {}
    ~CoutRedirect() { std::cout.rdbuf(old_); }
    CoutRedirect(const CoutRedirect&) = delete;
    CoutRedirect& operator=(const CoutRedirect&) = delete;
};

void example() {
    std::ofstream file("out.txt");
    CoutRedirect redirect(file.rdbuf());
    std::cout << "goes to the file\n";
} // Restored automatically – critical, since `file` dies here
```

Never let a redirected `cout` outlive the buffer it points at. RAII is the only safe pattern.

A Null Sink (discard everything)

```
#include <streambuf>
#include <ostream>

class NullBuf : public std::streambuf {
protected:
    int_type overflow(int_type c) override {
        return c; // Accept and drop; anything != eof means success
    }
    std::streamsize xsputn(const char*, std::streamsize n) override {
        return n; // Claim we wrote them all
    }
};

class NullStream : public std::ostream {
    NullBuf buf_;
public:
    NullStream() : std::ostream(&buf_) {}
};

// NullStream devnull;
// devnull << "expensive " << computation() << '\n'; // Formatting still runs
```

A Tee (write to two streams at once)

```
#include <streambuf>
#include <ostream>

class TeeBuf : public std::streambuf {
    std::streambuf* a_;
    std::streambuf* b_;
public:
    TeeBuf(std::streambuf* a, std::streambuf* b) : a_(a), b_(b) {}
protected:
    int_type overflow(int_type c) override {
        if (traits_type::eq_int_type(c, traits_type::eof())) return traits_type::not_eof(c);
        const auto ch = traits_type::to_char_type(c);
        const bool ok = !traits_type::eq_int_type(a_>sputc(ch), traits_type::eof())
            && !traits_type::eq_int_type(b_>sputc(ch), traits_type::eof());
        return ok ? c : traits_type::eof();
    }
    int sync() override {
        return (a_>pubsync() == 0 && b_>pubsync() == 0) ? 0 : -1;
    }
};

// std::ofstream file("log.txt");
// TeeBuf tee(std::cout.rdbuf(), file.rdbuf());
// std::ostream both(&tee);
// both << "appears in both places\n";
```

A Prefixing Buffer (timestamps, log levels, indentation)

```
#include <streambuf>
#include <ostream>
#include <string>

class PrefixBuf : public std::streambuf {
    std::streambuf* dest_;
    std::string prefix_;
    bool atLineStart_ = true;
public:
    PrefixBuf(std::streambuf* dest, std::string prefix)
        : dest_(dest), prefix_(std::move(prefix)) {}
protected:
    int_type overflow(int_type c) override {
        if (traits_type::eq_int_type(c, traits_type::eof()))
            return traits_type::not_eof(c);

        if (atLineStart_) {
            dest_>sputn(prefix_.data(), static_cast<std::streamsize>(prefix_.size()));
            atLineStart_ = false;
        }
        const auto ch = traits_type::to_char_type(c);
        atLineStart_ = (ch == '\n');
        return dest_>sputc(ch);
    }
    int sync() override { return dest_>pubsync(); }
};

// PrefixBuf pb(std::cout.rdbuf(), "[INFO] ");
// std::ostream log(&pb);
// log << "starting\nready\n";    // Both lines get the prefix
```

A Buffered Output Buffer (the realistic shape)

```
#include <streambuf>
#include <vector>
#include <algorithm>
```

```

class ChunkBuf : public std::streambuf {
    std::vector<char> buf_;
public:
    explicit ChunkBuf(std::size_t size = 4096) : buf_(size) {
        // Leave one slot so overflow's char always has somewhere to go
        setp(buf_.data(), buf_.data() + buf_.size() - 1);
    }
    ~ChunkBuf() override { sync(); }

protected:
    int_type overflow(int_type c) override {
        if (!traits_type::eq_int_type(c, traits_type::eof())) {
            *pptr() = traits_type::to_char_type(c);
            pbump(1);
        }
        return flushBuffer() ? traits_type::not_eof(c) : traits_type::eof();
    }
    int sync() override { return flushBuffer() ? 0 : -1; }

private:
    bool flushBuffer() {
        const std::ptrdiff_t n = pptr() - pbase();
        if (n > 0) {
            if (!writeToDevice(pbase(), n)) return false;
            pbump(static_cast<int>(-n)); // Reset pptr to pbase
        }
        return true;
    }
    bool writeToDevice(const char* data, std::ptrdiff_t n); // Your transport here
};

```

A Buffered Input Buffer

```

#include <streambuf>
#include <vector>
#include <cstring> // memmove
#include <algorithm> // min

class ChunkInBuf : public std::streambuf {
    std::vector<char> buf_;
    static constexpr std::size_t putbackSize = 8;
public:
    explicit ChunkInBuf(std::size_t size = 4096) : buf_(size + putbackSize) {
        char* end = buf_.data() + buf_.size();
        setg(end, end, end); // Empty get area → first read triggers underflow
    }

protected:
    int_type underflow() override {
        if (gptr() < egptr()) return traits_type::to_int_type(*gptr());

        // Preserve a few characters for putback
        char* base = buf_.data();
        char* start = base;
        std::size_t keep = 0;
        if (eback() == base) {
            keep = std::min<std::size_t>(putbackSize,
                                         static_cast<std::size_t>(gptr() - eback()));
            std::memmove(base, egptr() - keep, keep);
            start += keep;
        }

        const std::streamsize n = readFromDevice(start,
                                                  static_cast<std::streamsize>(buf_.size() - keep));
        if (n <= 0) return traits_type::eof();

        setg(base, start, start + n);
        return traits_type::to_int_type(*gptr());
    }
};

```

```
private:
    std::streamsize readFromDevice(char* dest, std::streamsize n); // Your transport here
};
```

Reading a Whole Stream via the Buffer

```
#include <sstream>
#include <fstream>

std::ifstream in("file.txt", std::ios::binary);
std::ostringstream ss;
ss << in.rdbuf();
std::string contents = ss.str();
```

Attaching a Stream to an Existing Buffer

```
#include <fstream>
#include <ostream>

std::filebuf fb;
if (fb.open("out.txt", std::ios::out)) {
    std::ostream os(&fb);
    os << "written through a raw filebuf\n";
    fb.close();
}
```

Character-Level Access

```
std::streambuf* sb = std::cin.rdbuf();

int_type c;
while ((c = sb->sgetc()) != std::char_traits<char>::eof()) {
    char ch = std::char_traits<char>::to_char_type(c);
    // Bypasses the stream layer entirely – no sentry, no formatting, no state flags.
    // Fast, but you also lose the stream's error reporting.
}
```

IMPORTANT CONCEPTS

Which Virtuals You Actually Need

For an **output-only** buffer: `overflow` (required) and `sync` (strongly recommended); `xspn` if you want a fast bulk path. For an **input-only** buffer: `underflow` (required); `pbackfail` if putback must work past the buffer start; `showmanyc` for non-blocking reads. Everything else has a usable default.

`overflow` Is Called When the Put Area Is Full — Or Always

If you never call `setp`, the put area is empty, so `overflow` is called for *every single character*. That is the simplest possible implementation (the null sink and tee above) and also the slowest. Calling `setp` with a real buffer makes `sputc` a pointer bump in the common case.

Reserve One Slot for `overflow`'s Argument

The conventional trick in `setp(begin, end - 1)` is to keep one byte free so `overflow` can store the character that triggered it before flushing. Without it you must flush first and then place the character, which is fine too — just be consistent.

`underflow` Does Not Advance; `uflow` Does

`underflow()` returns the next character and leaves `gptr()` pointing at it. The default `uflow()` calls `underflow()` then `gbump(1)`. Override only `underflow` unless your source cannot be re-read.

Return Values Are `int_type`, Not `char`

A `char` cannot represent EOF, so the buffer interface uses `int_type`. Always convert with `traits_type::to_int_type / to_char_type` and compare with `eq_int_type` — a raw `== EOF` breaks for `char` values like `0xFF` on platforms with signed `char`.

`not_eof` Is the Success Signal

`overflow` must return something that is not `eof()` to mean success. When called with `eof()` (which happens on flush), `traits_type::not_eof(c)` is the correct "nothing to do, but fine" answer.

Flush in the Destructor

A derived output buffer must flush in its destructor, or the tail of the output is silently lost. Note that the destructor cannot rely on virtual dispatch to a further-derived class — flush by calling a non-virtual private helper, as in the example above.

`rdbuf()` Does Not Transfer Ownership

`std::cout.rdbuf(other)` just swaps a pointer. The stream never deletes the buffer. If the buffer dies while a stream still points at it, every subsequent output is undefined behavior — hence the RAI redirect guard.

`pubsync()` VS `flush()`

`os.flush()` calls `rdbuf()->pubsync()`, which calls your `sync()`. Implementing `sync()` is what makes `std::endl` and `flush` actually reach your device.

Locale Awareness Is Optional

The `imbue` virtual lets a buffer react to locale changes — `filebuf` uses it to switch the `codecvt` facet for character encoding conversion. Most custom buffers can ignore it.

`std::spanbuf` and `std::syncbuf` Cover Common Needs

Before writing a custom buffer, check whether C++23's `spanbuf` (fixed external memory, no allocation) or C++20's `syncbuf` (thread-safe accumulation, atomic emit) already does what you want.

MINIMUM VIABLE BUFFER CHECKLIST

Output buffer	Input buffer
☐ <code>overflow(int_type)</code>	☐ <code>underflow()</code>
☐ <code>sync()</code>	☐ <code>setg()</code> in the constructor
☐ flush in the destructor	☐ <code>eof()</code> on exhaustion
☐ <code>setp()</code> if you want buffering	☐ putback area if putback matters
☐ <code>xspn()</code> for a bulk fast path	☐ <code>xsgetn()</code> for a bulk fast path
☐ non-copyable (base already is)	☐ <code>showmanyc()</code> if non-blocking

BEST PRACTICES

1. Prefer composing existing buffers — `filebuf`, `stringbuf`, `spanbuf`, `syncbuf` — over writing one
2. Use `os << in.rdbuf()` for whole-stream copies
3. Always guard `rdbuf()` swaps with `RAII` and restore before the buffer dies
4. Use `traits_type` helpers for every `char/int_type` conversion and comparison
5. Flush in your destructor, via a non-virtual helper
6. Implement `sync()` so `endl` and `flush` work as users expect
7. Call `setp/setg` with real buffers once correctness is established — per-character virtual calls are slow
8. Override `xspn / xsgetn` when your device has an efficient bulk path
9. Return `not_eof(c)`, not `c`, when `overflow` is called with `eof()`
10. Reserve putback space in input buffers if `unget / putback` must work
11. Keep buffers non-copyable — the base already deletes copy, don't re-enable it
12. Test the boundaries: exactly-full buffer, single-character writes, flush with an empty buffer, putback at the buffer start

RELATED HEADERS

```
#include <streambuf>    // basic_streambuf
#include <fstream>      // filebuf
#include <sstream>       // stringbuf
#include <spanstream>    // C++23: spanbuf – fixed external buffer
#include <syncstream>    // C++20: syncbuf – thread-safe output
#include <string>        // char_traits
#include <ios>           // ios_base, streamsize, openmode
#include <locale>        // codecvt, used by filebuf for encoding conversion
```

EXTERNAL RESOURCES

- `std::basic_streambuf`: https://en.cppreference.com/w/cpp/io/basic_streambuf
- `std::char_traits`: https://en.cppreference.com/w/cpp/string/char_traits
- **Standard C++ IOStreams and Locales** (Langer & Kreft) — the definitive treatment of custom buffers

Standard: C++11 baseline / C++20 (`syncbuf`) / C++23 (`spanbuf`) **Last Updated:** September 2026